MINI PROJECT REPORT

ON

NexStore – Web-Based E-Commerce Product Catalog and Feedback Management System



AT

5th Semester- Web Technology
Bachelor of Technology
in
Computer Science and Engineering

PREPARED BY

Aashish Jangid (IU2441230763)

UNDER GUIDANCE OF

Internal Guide

DHIVYA Vijayakannan
Assistant Professor,
Department of Computer Science and Engineering,
IITE, Indus University, Ahmedabad

SUBMITTED TO

INSTITUTE OF TECHNOLOGY AND ENGINEERING

INDUS UNIVERSITY CAMPUS, RANCHARDA, VIA-THALTEJ

AHMEDABAD-382115, GUJARAT, INDIA,

WEB: www.indusuni.ac.in

NOV 2026

CANDIDATE'S DECLARATION

I hereby declare that the Mini Project report entitled "**NexStore – Web-Based E-Commerce Product Catalog and Feedback Management System**" is the result of my own work carried out under the guidance and supervision of "**DHIVYA Vijayakannan**".

I further declare that, to the best of my knowledge, this report does not contain any part of the work that has been submitted previously for the award of any degree or diploma in this university or any other university, except where due acknowledgement and citation have been made.

_____

Candidate's Signature

AASHISH JANGID (IU2441230763)

_____

DHIVYA Vijayakannan
Assistant Professor
Department of Computer Science and Engineering,
Indus Institute of Technology and Engineering
INDUS UNIVERSITY– Ahmedabad,
State: Gujarat

INDUS INSTITUTE OF TECHNOLOGY AND ENGINEERING
COMPUTER SCIENCE & ENGINEERING
NOV 2026



CERTIFICATE

Date: __/10/2026

This is to certify that the Mini Project work entitled "**NexStore – Web-Based E-Commerce Product Catalog and Feedback Management System**" has been carried out by **AASHISH JANGID** under my guidance in partial fulfillment of the requirements for the **Bachelor of Technology** in **Computer Science and Engineering (5th Semester)** of **Indus University, Ahmedabad** during the academic year 2026.

DHIVYA Vijayakannan Assistant Professor, Department of Computer Science and Engineering, IITE, Indus University, Ahmedabad.	Dr. Kaushal Jani Head of Department, Department of Computer Science and Engineering, IITE, Indus University, Ahmedabad.	Prof. Zalak Vyas Head of Department, Department of Computer Science and Engineering, IITE, Indus University, Ahmedabad.

TABLE OF CONTENT

Title **Page No**

ABSTRACT.....**i**
LIST OF FIGURES.....**ii**
LIST OF TABLES.....**iii**
CHAPTER 1 INTRODUCTION.....**1**
CHAPTER 2 OBJECTIVE.....**4**
CHAPTER 3 PROPOSED WORK.....**7**
CHAPTER 4 TOOLS.....**10**
CHAPTER 5 SAMPLE CODE.....**13**
CHAPTER 6 OUTPUT SCREENSHOTS.....**29**
CONCLUSION.....**33**
BIBLIOGRAPHY.....**34**

ABSTRACT

NexStore is a web-based e-commerce product catalog developed as a Web Technology mini project. The system demonstrates the integration of HTML5, CSS, JavaScript, Tailwind CSS, PHP, PDO and MySQL to create an interactive product browsing and basic inventory-management application. Customers can search and filter products, view detailed information, like products, maintain a session-based wishlist, and submit feedback. A business-owner view allows products to be added and deleted from the catalog. The application communicates asynchronously with PHP endpoints using the Fetch API and stores catalog information in MySQL. The supplied source was also reviewed for consistency between the database schema and PHP code; a product_id field is required in the feedback table for the current review endpoints to work correctly. The project provides a practical foundation that can be extended with authentication, persistent user preferences, checkout, payment processing, inventory control and stronger security.

Keywords: NexStore, E-Commerce, HTML5, CSS, JavaScript, Tailwind CSS, PHP, PDO, MySQL, Fetch API

LIST OF FIGURES

Figure No	Title	Page No.
Figure 6.1	Home Page / Customer View	30
Figure 6.2	Product Detail Modal	30
Figure 6.3	Business Owner Dashboard	31
Figure 6.4	Seller Inventory and Delete Action	31
Figure 6.5	Database / phpMyAdmin – Products Table	32

LIST OF TABLES

Table No	Title	Page No.
Table 2.1	Functional Requirements	5
Table 3.1	Module Description	8
Table 3.2	Database Design	9
Table 4.1	Software and Technologies	11
Table 4.2	Hardware Requirements	11

CHAPTER 1
INTRODUCTION

- **BACKGROUND**
- **PROBLEM STATEMENT**
- **SCOPE AND SIGNIFICANCE**

1.1 BACKGROUND

Traditional product catalog systems often provide only static information, while small businesses need a simple web interface through which products can be displayed, searched, liked, saved to a wishlist, reviewed, added, and removed. NexStore is developed as a Web Technology mini project to demonstrate how HTML, CSS, JavaScript, PHP and MySQL can be combined to build an interactive database-driven web application. The system provides customer-facing product discovery and a business-owner management interface.

KEY FEATURES:

1. Product catalog: products are loaded dynamically from the database and displayed in responsive cards.
2. Search and filtering: users can search by product title or category and switch between all, wishlist, and liked views.
3. Product details: selecting a product opens a modal containing the image, price, category, description, likes, wishlist action, and feedback area.
4. Customer interaction: users can like products and maintain a client-side wishlist during the current browser session.
5. Seller management: the business-owner view provides a form to add products and an inventory list with delete actions.
6. Responsive design: Tailwind CSS utility classes and custom CSS are used to create a clean layout that adapts to different screen sizes.

1.2 PROBLEM STATEMENT

The purpose of NexStore is to provide a compact demonstration of an e-commerce-style product catalog with interactive front-end behavior and server-side data processing. The project focuses on product discovery, category filtering, search, product details, wishlist and like interactions, seller inventory management, and customer feedback. The implementation also demonstrates asynchronous communication between JavaScript and PHP using the Fetch API.

The project was selected because e-commerce interfaces are a practical way to demonstrate important Web Technology concepts. The application combines responsive UI design, DOM manipulation, event handling, AJAX-style requests, PHP server processing, prepared SQL statements, and relational data storage.

The application uses MySQL as the persistent data store. The supplied schema creates the shop database and tables for users, products, wishlist, and feedback. PHP scripts provide JSON responses for product retrieval, product creation, product

deletion, and feedback operations. JavaScript consumes these responses and updates the interface without requiring a complete page reload.

The system is intentionally kept lightweight so that the complete workflow can be understood within a Web Technology mini-project. It is not presented as a payment-enabled commercial platform. Authentication, secure seller authorization, checkout, payment processing, server-side like persistence, and a fully relational wishlist implementation are possible future enhancements.

1.3 SCOPE AND SIGNIFICANCE

1. To demonstrate an interactive e-commerce catalog that responds to user actions and communicates with a PHP/MySQL backend.
2. To implement product retrieval, search, category filtering, wishlist and like interactions, detailed product viewing, feedback submission, and seller-side product management using standard Web Technology techniques.
3. To separate the presentation, client-side interaction, server-side processing, and database layers so that each component has a clear responsibility.
4. To provide a responsive interface that works across common desktop and mobile viewport sizes.
5. To provide a foundation that can later support authentication, persistent customer preferences, checkout, and additional product-management functions.
6. To identify implementation limitations and document realistic future improvements.
7. To evaluate the use of HTML5, CSS, JavaScript, PHP, MySQL, Fetch API, Tailwind CSS, and Font Awesome in a single integrated mini-project.

CHAPTER 2
OBJECTIVE

- **GENERAL OBJECTIVES**
- **FUNCTIONAL REQUIREMENTS**
- **NON-FUNCTIONAL REQUIREMENTS**
- **PROJECT SCOPE**

2.1 GENERAL OBJECTIVES

The primary objective of NexStore is to develop a functional web-based product catalog that demonstrates the integration of client-side and server-side Web Technology components.

- Develop a responsive and visually organized product catalog interface.
- Retrieve product data from a MySQL database through PHP APIs.
- Implement client-side search and category-based product filtering.
- Provide customer interactions such as likes and wishlist selection.
- Display complete product information through a dynamic modal window.
- Provide a business-owner interface for adding and deleting product listings.
- Implement a feedback and rating workflow using PHP and MySQL.
- Demonstrate asynchronous browser-server communication using the Fetch API.
- Maintain a clear separation between HTML presentation, JavaScript behavior, PHP processing, and SQL data storage.

2.2 FUNCTIONAL REQUIREMENTS

Table 2.1 lists the main functional requirements of NexStore and what each of them does.

Table 2.1: Functional Requirements

Requirement	Description
Product listing	Display products with image, title, category, description, price and like count.
Search	Filter products by title or category as the user types.
Wishlist	Allow the user to save or remove products from a browser-session wishlist.
Like	Allow the user to like/unlike a product and update the visible count.
Product details	Open a modal containing detailed information and customer feedback.
Add product	Seller view submits product information to add_product.php.
Delete product	Seller view sends the product ID to delete_product.php.
Feedback	Collect name, email, rating and review message for a product.
Persistence	Store product data in MySQL and return records as JSON.

2.3 NON-FUNCTIONAL REQUIREMENTS

The interface should be responsive, easy to navigate, visually consistent, reasonably fast for a small catalog, and maintainable by separating the main responsibilities across files. Basic input validation is performed in both the browser and PHP endpoints.

2.4 PROJECT SCOPE

The current scope covers catalog browsing and basic seller-side catalog administration. It does not include payment gateway integration, order processing, production-grade authentication/authorization, image uploading, stock management, or deployment hardening.

CHAPTER 3
PROPOSED WORK

- **SYSTEM ARCHITECTURE**
- **MODULE DESCRIPTION**
- **DATABASE DESIGN**
- **DATA FLOW**
- **USER INTERFACE DESIGN**

3.1 SYSTEM ARCHITECTURE

NexStore follows a simple three-layer architecture. The presentation layer consists of index.html, style.css, Tailwind CSS and Font Awesome. The client logic layer is implemented in script.js and manages fetching, rendering, filtering, modal behavior, likes, wishlist state, and form submission. The server/data layer consists of PHP endpoints and MySQL accessed through PDO.

Browser → JavaScript Fetch API → PHP Endpoint → PDO → MySQL Database → JSON Response → Dynamic DOM Update

3.2 MODULE DESCRIPTION

Table 3.1: Module Description

Module	Main Files	Purpose
User Interface	index.html, style.css	Provides customer and business-owner views and responsive presentation.
Client Logic	script.js	Loads data, renders cards, filters products, controls modal windows and submits forms.
Database Connection	db.php	Creates a PDO connection to the shop_db MySQL database.
Product Retrieval	get_products.php	Returns all products ordered by descending ID as JSON.
Product Creation	add_product.php	Validates and inserts a new product.
Product Deletion	delete_product.php	Deletes a product by ID.
Feedback Retrieval	get_feedback.php	Retrieves feedback for a product.
Feedback Submission	submit_feedback.php	Validates and inserts feedback.
Database Schema	schema.sql	Creates database tables and sample product records.

3.3 DATABASE DESIGN

The supplied schema defines users, products, wishlist, and feedback tables. The products table is the core catalog table. The wishlist table references products using a foreign key and prevents duplicate user/product combinations. The users table is prepared for future authentication. The feedback definition supplied with the project, however, does not currently include the product_id column required by get_feedback.php and submit_feedback.php; this should be corrected before presenting feedback persistence as production-ready.

Table 3.2: Database Design

Table	Important Fields	Purpose
users	id, username, password, role, created_at	Prepared user and role structure.
products	id, title, price, category, image_url, description, likes_count, created_at	Stores product catalog data.
wishlist	id, user_id, product_id, created_at	Stores saved products; currently the UI uses client-side state.
feedback	id, name, email, rating, message, created_at	Stores reviews; should additionally contain product_id for current PHP queries.

3.4 DATA FLOW

- On page load, DOMContentLoaded calls fetchProducts().
- fetchProducts() requests get_products.php.
- The PHP endpoint queries products through PDO and returns JSON.
- JavaScript stores the returned array and renders customer and seller views.
- Search and view filters operate on the client-side product array.
- Seller add/delete actions call the relevant PHP endpoints and then refresh the catalog.
- Opening a product modal requests product-specific feedback from get_feedback.php.
- Feedback submission posts FormData to submit_feedback.php and refreshes the modal after success.

3.5 USER INTERFACE DESIGN

The customer interface contains a sticky header, role switcher, promotional banner, search field, filter controls, product grid, and dynamic product modal. The seller interface contains a management banner, add-product form, and inventory grid. Tailwind utility classes provide spacing, grid behavior, colors, borders, shadows, and responsive breakpoints.

CHAPTER 4
TOOLS

- **SOFTWARE AND TECHNOLOGIES**
- **HARDWARE REQUIREMENTS**
- **INSTALLATION AND EXECUTION**
- **SECURITY AND VALIDATION**

4.1 SOFTWARE AND TECHNOLOGIES

Table 4.1 lists the software and technologies used to build and run NexStore.

Table 4.1: Software and Technologies

Tool / Technology	Use in Project
HTML5	Defines page structure and forms.
CSS3	Provides custom transitions and product-card interaction styling.
JavaScript (ES6+)	Implements dynamic rendering, events, filtering and Fetch API communication.
Tailwind CSS	Provides responsive utility-first styling through the CDN.
Font Awesome	Provides interface icons.
PHP	Implements server-side endpoints and database operations.
MySQL	Stores products, users, wishlist and feedback records.
PDO	Provides prepared database access from PHP.
Fetch API	Provides asynchronous browser-to-server communication.
VS Code / similar editor	Suitable environment for editing and maintaining source files.
XAMPP / PHP-MySQL server	Suitable local environment for executing PHP and MySQL.

4.2 HARDWARE REQUIREMENTS

The minimum suggested hardware for running the project is given in Table 4.2.

Table 4.2: Hardware Requirements

Component	Minimum Suggested Requirement
Processor	Dual-core processor or better
RAM	4 GB or higher
Storage	At least 500 MB free for project and local server
Display	1366 × 768 or higher recommended
Network	Required for CDN and remote product images when internet-hosted assets are used

4.3 INSTALLATION AND EXECUTION

- Install a PHP/MySQL development environment such as XAMPP.
- Place the project directory inside the server document root, for example htdocs.
- Start Apache and MySQL services.

- Open phpMyAdmin and execute schema.sql to create shop_db and its tables.
- Update db.php if the local MySQL username, password, host, or database name differs.
- Open index.html through the PHP server URL rather than relying on file:// when testing PHP endpoints.
- Verify that product records are returned by get_products.php.
- Test customer interactions and seller operations from the browser.

4.4 SECURITY AND VALIDATION CONSIDERATIONS

The PHP endpoints use PDO prepared statements for database inserts, selects and deletes. Basic server-side validation is present for product title, price, product ID and feedback fields. The current implementation should still be strengthened with authentication, authorization, CSRF protection, stricter input validation, output escaping on every dynamic field, secure password hashing, rate limiting, and production-safe database credentials before deployment.

CHAPTER 5
SAMPLE CODE

- **FETCHING PRODUCTS**
- **ADDING A PRODUCT**
- **DELETING A PRODUCT**
- **SUBMITTING FEEDBACK**
- **GETTING FEEDBACK**
- **DATABASE CONNECTION**
- **HTML FILE**
- **STYLE SHEET**
- **JAVASCRIPT FILE**
- **SQL FILE**

5.1 FETCHING PRODUCTS (GET_PRODUCTS.PHP)

get_products.php returns all the products, ordered by descending ID, as JSON.

```
<?php
// get_products.php
header('Content-Type: application/json');
require 'db.php';

try {
    $stmt = $pdo->query("SELECT * FROM products ORDER BY id DESC");
    $products = $stmt->fetchAll();
    echo json_encode(["success" => true, "data" => $products]);
} catch (PDOException $e) {
    echo json_encode(["success" => false, "message" => $e->getMessage()]);
}
?>
```

5.2 ADDING A PRODUCT (ADD_PRODUCT.PHP)

add_product.php validates the product details sent by the seller view and inserts a new product.

```
<?php
// add_product.php
header('Content-Type: application/json');
require_once 'db.php';

try {
    // Read parameters from FormData or JSON payload
    $title       = $_POST['title'] ?? '';
    $price       = $_POST['price'] ?? 0;
    $category    = $_POST['category'] ?? 'Electronics';
    $image_url   = $_POST['image_url'] ?? '';
    $description = $_POST['description'] ?? '';

    if (empty($title)) {
        $raw = file_get_contents('php://input');
        $json = json_decode($raw, true);
        if ($json) {
            $title       = $json['title'] ?? '';
            $price       = $json['price'] ?? 0;
            $category    = $json['category'] ?? 'Electronics';
            $image_url   = $json['image_url'] ?? '';
            $description = $json['description'] ?? '';
        }
    }

    $title     = trim($title);
    $price     = floatval($price);
    $image_url = trim($image_url);

    if (empty($title) || $price <= 0) {
        echo json_encode(["success" => false, "message" => "Title and a valid
    Price are required."]);
        exit();
    }

    if (empty($image_url)) {
        $image_url = "https://images.unsplash.com/photo-1523275335684-
    37898b6baf30?auto=format&fit=crop&w=600&q=80";
    }
```

```
    $stmt = $pdo->prepare("INSERT INTO products (title, price, category,
    image_url, description) VALUES (?, ?, ?, ?, ?)");
    $stmt->execute([$title, $price, $category, $image_url, $description]);

    echo json_encode([
        "success" => true,
        "id" => $pdo->lastInsertId(),
        "message" => "Product saved successfully!"
    ]);

} catch (PDOException $e) {
    http_response_code(500);
    echo json_encode(["success" => false, "message" => "Database Error: " . $e-
    >getMessage()]);
} catch (Exception $e) {
    http_response_code(500);
    echo json_encode(["success" => false, "message" => "Server Error: " . $e-
    >getMessage()]);
}
?>
```

5.3 DELETING A PRODUCT (DELETE_PRODUCT.PHP)

delete_product.php deletes a product from the catalog using its ID.

```
<?php
// delete_product.php
header('Content-Type: application/json');
require_once 'db.php';

try {
    $product_id = intval($_POST['product_id'] ?? 0);

    if ($product_id <= 0) {
        // Fallback for JSON body
        $raw = file_get_contents('php://input');
        $json = json_decode($raw, true);
        $product_id = intval($json['product_id'] ?? 0);
    }

    if ($product_id <= 0) {
        echo json_encode(["success" => false, "message" => "Invalid Product
    ID."]);
        exit();
    }

    $stmt = $pdo->prepare("DELETE FROM products WHERE id = ?");
    $stmt->execute([$product_id]);

    echo json_encode(["success" => true, "message" => "Product deleted
    successfully from catalog."]);
} catch (Exception $e) {
    http_response_code(500);
    echo json_encode(["success" => false, "message" => "Server Error: " . $e-
    >getMessage()]);
}
?>
```

5.4 SUBMITTING FEEDBACK (SUBMIT_FEEDBACK.PHP)

submit_feedback.php validates the review details and saves the feedback for a product.

```
<?php
// submit_feedback.php
header('Content-Type: application/json');
require_once 'db.php';

try {
    $product_id = intval($_POST['product_id'] ?? 0);
    $name       = trim($_POST['name'] ?? '');
    $email      = trim($_POST['email'] ?? '');
    $rating     = intval($_POST['rating'] ?? 5);
    $message    = trim($_POST['message'] ?? '');

    if ($product_id <= 0 || empty($name) || empty($email) || empty($message)) {
        echo json_encode(["success" => false, "message" => "Product ID, Name,
    Email, and Message are required."]);
        exit();
    }

    $stmt = $pdo->prepare("INSERT INTO feedback (product_id, name, email, rating,
    message) VALUES (?, ?, ?, ?, ?)");
    $stmt->execute([$product_id, $name, $email, $rating, $message]);

    echo json_encode([
        "success" => true,
        "message" => "Thank you! Your feedback has been saved for this product."
    ]);

} catch (Exception $e) {
    http_response_code(500);
    echo json_encode(["success" => false, "message" => "Server Error: " . $e-
    >getMessage()]);
}
?>
```

5.5 GETTING FEEDBACK (GET_FEEDBACK.PHP)

get_feedback.php retrieves the feedback saved for a given product.

```
<?php
// get_feedback.php
header('Content-Type: application/json');
require_once 'db.php';

try {
    $product_id = intval($_GET['product_id'] ?? 0);

    if ($product_id <= 0) {
        echo json_encode(["success" => false, "message" => "Valid product_id is
    required."]);
        exit();
    }

    $stmt = $pdo->prepare("SELECT name, rating, message, created_at FROM feedback
    WHERE product_id = ? ORDER BY id DESC");
    $stmt->execute([$product_id]);
    $reviews = $stmt->fetchAll();

    echo json_encode(["success" => true, "data" => $reviews]);
} catch (Exception $e) {
```

```
    http_response_code(500);
    echo json_encode(["success" => false, "message" => "Server Error: " . $e-
    >getMessage()]);
}
?>
```

5.6 DATABASE CONNECTION (DB.PHP)

db.php creates the PDO connection to the shop_db MySQL database.

```
<?php
// db.php
error_reporting(E_ALL);
ini_set('display_errors', 0);

$host   = "localhost";
$user   = "root";
$pass   = "";
$dbname = "shop_db";

try {
    $pdo = new PDO("mysql:host=$host;dbname=$dbname;charset=utf8mb4", $user,
    $pass, [
        PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
        PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC
    ]);
} catch (PDOException $e) {
    header('Content-Type: application/json');
    http_response_code(500);
    echo json_encode([
        "success" => false,
        "message" => "Database Connection Failed: " . $e->getMessage()
    ]);
    exit();
}
?>
```

5.7 HTML FILE (INDEX.HTML)

index.html provides the customer view and the business-owner view of the application.

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>NexStore - E-Commerce Platform</title>
    <!-- Tailwind CSS -->
    <script src="https://cdn.tailwindcss.com"></script>
    <link rel="stylesheet" href="style.css">
    <!-- FontAwesome Icons -->
    <link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-
    awesome/6.4.0/css/all.min.css">
</head>
<body class="bg-gray-50 font-sans text-gray-800 antialiased min-h-screen flex
    flex-col">

    <!-- Header -->
    <header class="bg-white shadow-sm border-b sticky top-0 z-30">
        <div class="max-w-7xl mx-auto px-4 py-3 flex justify-between items-
    center">
```

```
        <div class="flex items-center gap-3 cursor-pointer"
onclick="filterCatalog('all')">
            <div class="bg-indigo-600 text-white p-2 rounded-lg font-bold
text-xl"><i class="fa-solid fa-store"></i></div>
            <h1 class="font-bold text-xl text-gray-900">NexStore</h1>
        </div>

        <!-- Role Switcher -->
        <div class="flex items-center gap-4">
            <div class="bg-gray-100 p-1 rounded-xl flex items-center text-sm
font-semibold">
                <button id="role-customer-btn"
onclick="switchRole('customer')" class="px-4 py-1.5 rounded-lg bg-white
shadow-sm text-indigo-600 transition">
                    <i class="fa-solid fa-user mr-1"></i> Customer View
                </button>
                <button id="role-seller-btn" onclick="switchRole('seller')"
class="px-4 py-1.5 rounded-lg text-gray-600 transition">
                    <i class="fa-solid fa-store mr-1"></i> Business Owner View
                </button>
            </div>
        </div>
    </div>
</header>

<!-- Main Content -->
<main class="max-w-7xl mx-auto px-4 py-8 flex-grow w-full">

    <!-- ================= CUSTOMER VIEW ================= -->
    <section id="customer-view" class="space-y-8">
        <div class="bg-indigo-600 text-white p-8 rounded-2xl shadow-lg flex
justify-between items-center">
            <div>
                <h2 class="text-3xl font-bold mb-2">Discover Premium
Products</h2>
                <p class="text-indigo-100">Click any product to view high-res
image, full details, saved reviews, and leave feedback!</p>
            </div>
        </div>

        <!-- Search & Filters -->
        <div class="flex flex-col sm:flex-row justify-between items-center
gap-4 bg-white p-4 rounded-xl border shadow-sm">
            <input type="text" id="search-input" onkeyup="filterProducts()"
placeholder="Search products..." class="w-full sm:w-72 px-4 py-2 border
rounded-lg focus:outline-none focus:ring-2 focus:ring-indigo-500">

            <div class="flex items-center gap-2">
                <button onclick="filterCatalog('all')" id="filter-all-btn"
class="px-4 py-2 rounded-lg text-sm font-semibold bg-indigo-600 text-white
transition shadow-sm">
                    All Items
                </button>
                <button onclick="filterCatalog('wishlist')" id="filter-
wishlist-btn" class="px-4 py-2 rounded-lg text-sm font-semibold bg-gray-100
text-gray-700 hover:bg-indigo-50 transition">
                    <i class="fa-solid fa-bookmark text-indigo-600 mr-1"></i>
Wishlist (<span id="wishlist-count">0</span>)
                </button>
                <button onclick="filterCatalog('liked')" id="filter-liked-btn"
class="px-4 py-2 rounded-lg text-sm font-semibold bg-gray-100 text-gray-700
hover:bg-red-50 transition">
                    <i class="fa-solid fa-heart text-red-500 mr-1"></i> Liked
(<span id="liked-count">0</span>)
                </button>
            </div>
        </div>
```

```
        <div id="view-title-banner" class="text-xl font-bold text-gray-800
flex items-center justify-between">
            <span id="view-title-text">All Products</span>
        </div>

        <div id="products-grid" class="grid grid-cols-1 sm:grid-cols-2
lg:grid-cols-3 gap-6">
            <!-- Loaded via JS -->
        </div>
    </section>

    <!-- ================ SELLER / BUSINESS OWNER VIEW ================ -->
    <section id="seller-view" class="hidden space-y-8">
        <div class="bg-slate-800 text-white p-8 rounded-2xl shadow-lg">
            <h2 class="text-3xl font-bold mb-2">Business Owner Management
Dashboard</h2>
            <p class="text-slate-300">Add new products to your shop or delete
existing inventory listings.</p>
        </div>

        <!-- Add Product Form -->
        <div class="bg-white p-6 rounded-2xl border shadow-sm max-w-3xl mx-
auto">
            <h3 class="text-xl font-bold mb-4 flex items-center gap-2"><i
class="fa-solid fa-plus-circle text-emerald-600"></i> Add New Item</h3>
            <form id="add-product-form"
onsubmit="handleAddProductSubmit(event)" class="space-y-4">
                <div class="grid grid-cols-1 sm:grid-cols-2 gap-4">
                    <div>
                        <label class="block text-sm font-medium text-gray-
700">Product Title</label>
                        <input type="text" name="title" required
placeholder="e.g., Wireless Mouse" class="w-full mt-1 border px-3 py-2
rounded-lg">
                    </div>
                    <div>
                        <label class="block text-sm font-medium text-gray-
700">Price ($)</label>
                        <input type="number" step="0.01" name="price" required
placeholder="29.99" class="w-full mt-1 border px-3 py-2 rounded-lg">
                    </div>
                </div>
                <div class="grid grid-cols-1 sm:grid-cols-2 gap-4">
                    <div>
                        <label class="block text-sm font-medium text-gray-
700">Category</label>
                        <select name="category" class="w-full mt-1 border px-3
py-2 rounded-lg">
                            <option value="Electronics">Electronics</option>
                            <option value="Home & Living">Home &
Living</option>
                            <option value="Fashion">Fashion</option>
                        </select>
                    </div>
                    <div>
                        <label class="block text-sm font-medium text-gray-
700">Image URL</label>
                        <input type="url" name="image_url" required
placeholder="https://..." class="w-full mt-1 border px-3 py-2 rounded-lg">
                    </div>
                </div>
                <div>
                    <label class="block text-sm font-medium text-gray-
700">Full Description</label>
                    <textarea name="description" rows="3"
placeholder="Detailed product description..." class="w-full mt-1 border px-3
py-2 rounded-lg"></textarea>
                </div>
```

```
                    <button type="submit" class="w-full bg-emerald-600 text-white
    py-2.5 rounded-lg font-semibold hover:bg-emerald-700 transition">Publish
    Product</button>
                </form>
            </div>

            <!-- Seller Inventory List with Delete Action -->
            <div class="bg-white p-6 rounded-2xl border shadow-sm">
                <h3 class="text-xl font-bold mb-4 flex items-center gap-2"><i
    class="fa-solid fa-boxes-stacked text-indigo-600"></i> Manage Catalog
    Listings</h3>
                <div id="seller-inventory-grid" class="grid grid-cols-1 sm:grid-
    cols-2 lg:grid-cols-3 gap-6">
                    <!-- Loaded via JS -->
                </div>
            </div>
        </section>

    </main>

    <!-- Product Detail & Reviews Modal -->
    <div id="product-modal" class="fixed inset-0 bg-black/60 backdrop-blur-sm z-50
    flex items-center justify-center hidden p-4 overflow-y-auto">
        <div class="bg-white rounded-2xl max-w-4xl w-full overflow-hidden shadow-
    2xl relative my-8 max-h-[90vh] flex flex-col">
            <button onclick="closeProductModal()" class="absolute top-4 right-4
    bg-gray-100 hover:bg-gray-200 text-gray-700 w-10 h-10 rounded-full flex items-
    center justify-center font-bold z-10 transition">
                <i class="fa-solid fa-xmark text-lg"></i>
            </button>

            <div id="modal-content" class="overflow-y-auto p-6 space-y-6">
                <!-- Dynamically populated product details, feedback form, and
    existing reviews -->
            </div>
        </div>
    </div>

    <script src="script.js"></script>
</body>
</html>
```

5.8 STYLE SHEET (STYLE.CSS)

style.css adds the custom transitions and the product-card styling.

```
/* style.css */
body {
    transition: background-color 0.3s ease;
}

.product-card {
    transition: transform 0.2s ease, box-shadow 0.2s ease;
}

.product-card:hover {
    transform: translateY(-4px);
    box-shadow: 0 10px 25px -5px rgba(0, 0, 0, 0.1);
}

.liked-btn {
    color: #ef4444 !important;
}

.wishlist-btn-active {
    background-color: #4f46e5 !important;
```

```
    color: white !important;
}
```

5.9 JAVASCRIPT FILE (SCRIPT.JS)

script.js loads the data, renders the product cards, filters products, controls the modal window and submits the forms.

```
// script.js
let products = [];
let wishlist = new Set();
let likedProducts = new Set();
let currentFilter = 'all';

document.addEventListener("DOMContentLoaded", () => {
    fetchProducts();
});

// Fetch catalog from PHP backend
async function fetchProducts() {
    try {
        const response = await fetch('get_products.php');
        const data = await response.json();
        if (data.success) {
            products = data.data;
            applyCurrentFilter();
            renderSellerInventory();
        } else {
            console.error("Failed to load products:", data.message);
        }
    } catch (err) {
        console.error("Error connecting to server:", err);
    }
}

// Render Products Grid for Customers
function renderProducts(items) {
    const grid = document.getElementById('products-grid');
    grid.innerHTML = '';

    if (items.length === 0) {
        grid.innerHTML = `
            <div class="col-span-full text-center py-12 bg-white rounded-2xl
    border">
                <i class="fa-solid fa-box-open text-4xl text-gray-300 mb-3"></i>
                <p class="text-gray-500 font-medium">No items found in this
    view.</p>
            </div>`;
        return;
    }

    items.forEach(product => {
        const isLiked = likedProducts.has(product.id);
        const isWishlisted = wishlist.has(product.id);

        grid.innerHTML += `
            <div class="bg-white rounded-2xl border shadow-sm overflow-hidden flex
    flex-col hover:shadow-md transition">
                <div class="relative cursor-pointer" onclick="openProductModal($
    {product.id})">
                    <img src="${product.image_url}" alt="${product.title}"
    class="w-full h-56 object-cover">
                    <span class="absolute top-3 left-3 text-xs font-bold uppercase
    text-indigo-700 bg-white/90 backdrop-blur px-2.5 py-1 rounded-md shadow-sm">$
    {product.category}</span>
```

```
                </div>

                <div class="p-5 flex-grow flex flex-col justify-between">
                    <div>
                        <h4 onclick="openProductModal(${product.id})" class="font-
    bold text-lg text-gray-900 cursor-pointer hover:text-indigo-600 transition">$
    {product.title}</h4>
                        <p class="text-sm text-gray-500 mt-1 line-clamp-2">$
    {product.description || 'Click to view full details...'}</p>
                    </div>

                    <div class="mt-4 pt-3 border-t flex items-center justify-
    between">
                        <span class="text-xl font-bold text-gray-900">$$
    {parseFloat(product.price).toFixed(2)}</span>

                        <div class="flex items-center gap-2">
                            <button onclick="toggleLike(${product.id}, event)"
    class="p-2 rounded-lg text-sm font-semibold transition ${isLiked ? 'text-red-
    500 bg-red-50' : 'text-gray-500 hover:bg-gray-100'}">
                                <i class="fa-solid fa-heart"></i> <span>$
    {product.likes_count || 0}</span>
                            </button>
                            <button onclick="toggleWishlist(${product.id}, event)"
    class="p-2 rounded-lg text-sm font-semibold transition ${isWishlisted ? 'text-
    indigo-600 bg-indigo-50' : 'text-gray-500 hover:bg-gray-100'}">
                                <i class="fa-solid fa-bookmark"></i>
                            </button>
                            <button onclick="openProductModal(${product.id})"
    class="px-3 py-1.5 bg-indigo-600 text-white text-xs font-bold rounded-lg
    hover:bg-indigo-700 transition">
                                View Details
                            </button>
                        </div>
                    </div>
                </div>
            </div>
        `;
    });
}

// Render Seller Inventory Grid with Delete Buttons
function renderSellerInventory() {
    const grid = document.getElementById('seller-inventory-grid');
    if (!grid) return;
    grid.innerHTML = '';

    if (products.length === 0) {
        grid.innerHTML = `<p class="col-span-full text-center text-gray-500 py-
    8">No products listed in store.</p>`;
        return;
    }

    products.forEach(product => {
        grid.innerHTML += `
            <div class="bg-white rounded-xl border shadow-sm p-4 flex flex-col
    justify-between">
                <div class="flex gap-4">
                    <img src="${product.image_url}" alt="${product.title}"
    class="w-20 h-20 object-cover rounded-lg border">
                    <div>
                        <span class="text-xs font-semibold text-indigo-600
    uppercase">${product.category}</span>
                        <h4 class="font-bold text-gray-900">${product.title}</h4>
                        <p class="text-indigo-600 font-bold mt-1">$$
    {parseFloat(product.price).toFixed(2)}</p>
                    </div>
                </div>
```

```
                <p class="text-xs text-gray-500 mt-3 line-clamp-2">${product.description || 'No description'}</p>

                <div class="mt-4 pt-3 border-t flex justify-between items-center">
                    <span class="text-xs text-gray-400">ID: #${product.id}</span>
                    <button onclick="deleteProduct(${product.id})" class="px-3 py-1.5 bg-red-600 text-white text-xs font-bold rounded-lg hover:bg-red-700 transition flex items-center gap-1">
                        <i class="fa-solid fa-trash"></i> Delete Item
                    </button>
                </div>
            </div>
        `;
    });
}

// Open Detailed Product Modal & Load Saved Reviews
async function openProductModal(productId) {
    const product = products.find(p => p.id == productId);
    if (!product) return;

    const isLiked = likedProducts.has(product.id);
    const isWishlisted = wishlist.has(product.id);

    // Fetch stored feedback for this specific product
    let feedbackListHTML = `<p class="text-sm text-gray-500 italic">Loading feedback...</p>`;
    try {
        const res = await fetch(`get_feedback.php?product_id=${productId}`);
        const fbData = await res.json();

        if (fbData.success && fbData.data.length > 0) {
            feedbackListHTML = fbData.data.map(review => `
                <div class="p-3 bg-white border rounded-xl space-y-1 shadow-sm">
                    <div class="flex justify-between items-center">
                        <span class="font-bold text-sm text-gray-900">${escapeHTML(review.name)}</span>
                        <span class="text-xs text-yellow-500 font-bold">${'⭐'.repeat(review.rating)}</span>
                    </div>
                    <p class="text-sm text-gray-700">${escapeHTML(review.message)}</p>
                    <span class="text-[10px] text-gray-400">${review.created_at}</span>
                </div>
            `).join('');
        } else {
            feedbackListHTML = `<p class="text-sm text-gray-400 italic">No feedback submitted for this item yet. Be the first to review!</p>`;
        }
    } catch (err) {
        feedbackListHTML = `<p class="text-sm text-red-500">Could not load feedback.</p>`;
    }

    const modalContent = document.getElementById('modal-content');
    modalContent.innerHTML = `
        <div class="grid grid-cols-1 md:grid-cols-2 gap-8">
            <div>
                <img src="${product.image_url}" alt="${product.title}" class="w-full h-80 object-cover rounded-2xl shadow-sm border">
            </div>

            <div class="flex flex-col justify-between space-y-4">
                <div>
                    <span class="text-xs font-bold uppercase text-indigo-600 bg-indigo-50 px-2.5 py-1 rounded-md">${product.category}</span>
                    <h2 class="text-2xl font-bold text-gray-900 mt-2">${product.title}</h2>
```

```
                <p class="text-2xl font-extrabold text-indigo-600 mt-2">$$
{parseFloat(product.price).toFixed(2)}</p>
                <p class="text-gray-600 mt-4 leading-relaxed text-sm">$
{product.description || 'No detailed description provided.'}</p>
            </div>

            <div class="flex items-center gap-3 pt-4 border-t">
                <button onclick="toggleLike(${product.id}, event);
openProductModal(${product.id});" class="flex-1 py-2.5 px-4 rounded-xl text-sm
font-semibold border flex items-center justify-center gap-2 transition $
{isLiked ? 'bg-red-50 text-red-500 border-red-200' : 'text-gray-700 hover:bg-
gray-50'}">
                    <i class="fa-solid fa-heart"></i> ${isLiked ? 'Liked' :
'Like Item'} (${product.likes_count || 0})
                </button>
                <button onclick="toggleWishlist(${product.id}, event);
openProductModal(${product.id});" class="flex-1 py-2.5 px-4 rounded-xl text-sm
font-semibold border flex items-center justify-center gap-2 transition $
{isWishlisted ? 'bg-indigo-600 text-white' : 'text-gray-700 hover:bg-gray-
50'}">
                    <i class="fa-solid fa-bookmark"></i> ${isWishlisted ? 'In
Wishlist' : 'Add to Wishlist'}
                </button>
            </div>
        </div>
    </div>

    <!-- Feedback & Review Display Section -->
    <div class="pt-6 border-t space-y-6">
        <div>
            <h3 class="text-lg font-bold text-gray-900 mb-3 flex items-center
gap-2">
                <i class="fa-solid fa-comments text-indigo-600"></i> Customer
Feedback for this Product
            </h3>
            <div class="space-y-3 max-h-48 overflow-y-auto pr-2">
                ${feedbackListHTML}
            </div>
        </div>

        <div class="bg-gray-50 p-4 rounded-xl border space-y-3">
            <h4 class="text-sm font-bold text-gray-800">Leave Your Review</h4>
            <form onsubmit="handleProductFeedbackSubmit(event, ${product.id})"
class="space-y-3">
                <div class="grid grid-cols-1 sm:grid-cols-2 gap-3">
                    <input type="text" name="name" required placeholder="Your
Name" class="border px-3 py-2 rounded-lg text-sm bg-white">
                    <input type="email" name="email" required
placeholder="Your Email" class="border px-3 py-2 rounded-lg text-sm bg-white">
                </div>
                <div class="grid grid-cols-1 sm:grid-cols-3 gap-3">
                    <select name="rating" class="border px-3 py-2 rounded-lg
text-sm bg-white">
                        <option value="5">⭐⭐⭐⭐⭐ (5 Stars)</option>
                        <option value="4">⭐⭐⭐⭐ (4 Stars)</option>
                        <option value="3">⭐⭐⭐ (3 Stars)</option>
                        <option value="2">⭐⭐ (2 Stars)</option>
                        <option value="1">⭐ (1 Star)</option>
                    </select>
                    <input type="text" name="message" required
placeholder="Write your review..." class="sm:col-span-2 border px-3 py-2
rounded-lg text-sm bg-white">
                </div>
                <button type="submit" class="w-full bg-indigo-600 text-white
py-2 rounded-lg text-sm font-semibold hover:bg-indigo-700 transition">
                    Submit Feedback
                </button>
            </form>
        </div>
```

```
        </div>
    `;

    document.getElementById('product-modal').classList.remove('hidden');
}

function closeProductModal() {
    document.getElementById('product-modal').classList.add('hidden');
}

// Delete Product Action for Business Owners
async function deleteProduct(productId) {
    if (!confirm("Are you sure you want to delete this product?")) return;

    try {
        const formData = new FormData();
        formData.append('product_id', productId);

        const response = await fetch('delete_product.php', {
            method: 'POST',
            body: formData
        });

        const result = await response.json();
        if (result.success) {
            alert(result.message);
            fetchProducts(); // Reload inventory
        } else {
            alert("Error: " + result.message);
        }
    } catch (err) {
        alert("Server error occurred while deleting product.");
    }
}

// Navigation View Filters
function filterCatalog(type) {
    currentFilter = type;

    const btnAll = document.getElementById('filter-all-btn');
    const btnWishlist = document.getElementById('filter-wishlist-btn');
    const btnLiked = document.getElementById('filter-liked-btn');
    const bannerText = document.getElementById('view-title-text');

    btnAll.className = type === 'all' ? 'px-4 py-2 rounded-lg text-sm font-
    semibold bg-indigo-600 text-white transition shadow-sm' : 'px-4 py-2 rounded-
    lg text-sm font-semibold bg-gray-100 text-gray-700 hover:bg-indigo-50
    transition';
    btnWishlist.className = type === 'wishlist' ? 'px-4 py-2 rounded-lg text-sm
    font-semibold bg-indigo-600 text-white transition shadow-sm' : 'px-4 py-2
    rounded-lg text-sm font-semibold bg-gray-100 text-gray-700 hover:bg-indigo-50
    transition';
    btnLiked.className = type === 'liked' ? 'px-4 py-2 rounded-lg text-sm font-
    semibold bg-red-500 text-white transition shadow-sm' : 'px-4 py-2 rounded-lg
    text-sm font-semibold bg-gray-100 text-gray-700 hover:bg-red-50 transition';

    if (type === 'all') bannerText.innerText = "All Products";
    if (type === 'wishlist') bannerText.innerText = "Your Saved Wishlist Items";
    if (type === 'liked') bannerText.innerText = "Your Liked Products";

    applyCurrentFilter();
}

function applyCurrentFilter() {
    let filtered = products;

    if (currentFilter === 'wishlist') {
        filtered = products.filter(p => wishlist.has(p.id));
    } else if (currentFilter === 'liked') {
```

```
        filtered = products.filter(p => likedProducts.has(p.id));
    }

    renderProducts(filtered);
}

function toggleWishlist(productId, event) {
    if (event) event.stopPropagation();

    if (wishlist.has(productId)) {
        wishlist.delete(productId);
    } else {
        wishlist.add(productId);
    }

    document.getElementById('wishlist-count').innerText = wishlist.size;
    applyCurrentFilter();
}

function toggleLike(productId, event) {
    if (event) event.stopPropagation();

    const product = products.find(p => p.id == productId);
    if (!product) return;

    if (likedProducts.has(productId)) {
        likedProducts.delete(productId);
        product.likes_count--;
    } else {
        likedProducts.add(productId);
        product.likes_count++;
    }

    document.getElementById('liked-count').innerText = likedProducts.size;
    applyCurrentFilter();
}

function filterProducts() {
    const query = document.getElementById('search-input').value.toLowerCase();
    const filtered = products.filter(p => p.title.toLowerCase().includes(query) ||
    p.category.toLowerCase().includes(query));
    renderProducts(filtered);
}

async function handleProductFeedbackSubmit(event, productId) {
    event.preventDefault();
    const form = event.target;
    const formData = new FormData(form);
    formData.append('product_id', productId);

    try {
        const response = await fetch('submit_feedback.php', {
            method: 'POST',
            body: formData
        });
        const result = await response.json();

        if (result.success) {
            alert(result.message);
            form.reset();
            openProductModal(productId); // Refresh modal to show newly saved
    feedback
        } else {
            alert("Error: " + result.message);
        }
    } catch (err) {
        alert("Server error occurred while submitting feedback.");
    }
}
```

```
async function handleAddProductSubmit(event) {
    event.preventDefault();
    const form = event.target;
    const formData = new FormData(form);

    try {
        const response = await fetch('add_product.php', {
            method: 'POST',
            body: formData
        });
        const result = await response.json();

        if (result.success) {
            alert(result.message);
            form.reset();
            fetchProducts();
            switchRole('seller');
        } else {
            alert("Error: " + result.message);
        }
    } catch (err) {
        alert("Server error occurred while publishing product.");
    }
}

function switchRole(role) {
    const customerView = document.getElementById('customer-view');
    const sellerView = document.getElementById('seller-view');
    const customerBtn = document.getElementById('role-customer-btn');
    const sellerBtn = document.getElementById('role-seller-btn');

    if (role === 'customer') {
        customerView.classList.remove('hidden');
        sellerView.classList.add('hidden');
        customerBtn.className = "px-4 py-1.5 rounded-lg bg-white shadow-sm text-
    indigo-600 transition font-semibold";
        sellerBtn.className = "px-4 py-1.5 rounded-lg text-gray-600 transition
    font-semibold";
    } else {
        sellerView.classList.remove('hidden');
        customerView.classList.add('hidden');
        sellerBtn.className = "px-4 py-1.5 rounded-lg bg-white shadow-sm text-
    indigo-600 transition font-semibold";
        customerBtn.className = "px-4 py-1.5 rounded-lg text-gray-600 transition
    font-semibold";
        renderSellerInventory();
    }
}

function escapeHTML(str) {
    return String(str).replace(/[&<>"']/g, function (m) {
        return {
            '&': '&amp;',
            '<': '&lt;',
            '>': '&gt;',
            '"': '&quot;',
            "'": '&#039;'
        }[m];
    });
}
```

5.10 SQL FILE (SCHEMA.SQL)

schema.sql creates the shop_db database, its tables and the sample product records.

```
CREATE DATABASE IF NOT EXISTS shop_db;
USE shop_db;

-- Users Table
CREATE TABLE IF NOT EXISTS users (
    id INT AUTO_INCREMENT PRIMARY KEY,
    username VARCHAR(50) NOT NULL UNIQUE,
    password VARCHAR(255) NOT NULL,
    role ENUM('customer', 'seller') NOT NULL DEFAULT 'customer',
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Products Table
CREATE TABLE IF NOT EXISTS products (
    id INT AUTO_INCREMENT PRIMARY KEY,
    title VARCHAR(150) NOT NULL,
    price DECIMAL(10, 2) NOT NULL,
    category VARCHAR(50) NOT NULL,
    image_url TEXT NOT NULL,
    description TEXT,
    likes_count INT DEFAULT 0,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Wishlist Table
CREATE TABLE IF NOT EXISTS wishlist (
    id INT AUTO_INCREMENT PRIMARY KEY,
    user_id INT NOT NULL DEFAULT 1,
    product_id INT NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (product_id) REFERENCES products(id) ON DELETE CASCADE,
    UNIQUE(user_id, product_id)
);

-- Feedback Table
CREATE TABLE IF NOT EXISTS feedback (
    id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(100) NOT NULL,
    rating INT CHECK (rating >= 1 AND rating <= 5),
    message TEXT NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Insert Sample Products
INSERT INTO products (title, price, category, image_url, description, likes_count)
    VALUES
('Wireless ANC Headphones', 199.99, 'Electronics',
    'https://images.unsplash.com/photo-1505740420928-5e560c06d30e?
    auto=format&fit=crop&w=600&q=80', 'Premium noise-canceling headphones with
    30hr battery.', 12),
('Ergonomic Office Chair', 249.50, 'Home & Living',
    'https://images.unsplash.com/photo-1580481072645-022f9a6d1270?
    auto=format&fit=crop&w=600&q=80', 'Breathable mesh back with full lumbar
    support.', 8),
('Minimalist Smartwatch', 149.00, 'Electronics',
    'https://images.unsplash.com/photo-1523275335684-37898b6baf30?
    auto=format&fit=crop&w=600&q=80', 'Sleek design with OLED display and heart
    rate monitoring.', 19);
```

CHAPTER 6
OUTPUT SCREENSHOTS

- **HOME PAGE / CUSTOMER VIEW**
- **PRODUCT DETAIL MODAL**
- **BUSINESS OWNER DASHBOARD**
- **SELLER INVENTORY AND DELETE ACTION**
- **DATABASE / PHPMYADMIN – PRODUCTS TABLE**

This chapter shows the output screens of NexStore captured from the running project. They cover the customer view, the product detail window, the business-owner dashboard, the inventory list with the delete action and the database tables in phpMyAdmin.

6.1 HOME PAGE / CUSTOMER VIEW

The home page shows the NexStore header, the search box, the filter buttons and the product cards.

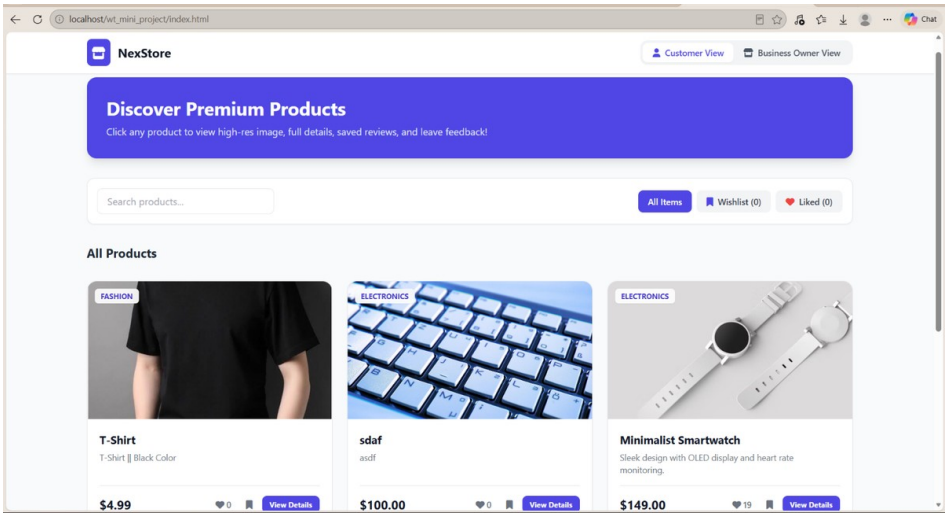


Figure 6.1: Home Page / Customer View

6.2 PRODUCT DETAIL MODAL

Clicking a product opens a window with its image, price, description, like and wishlist buttons and the feedback form.

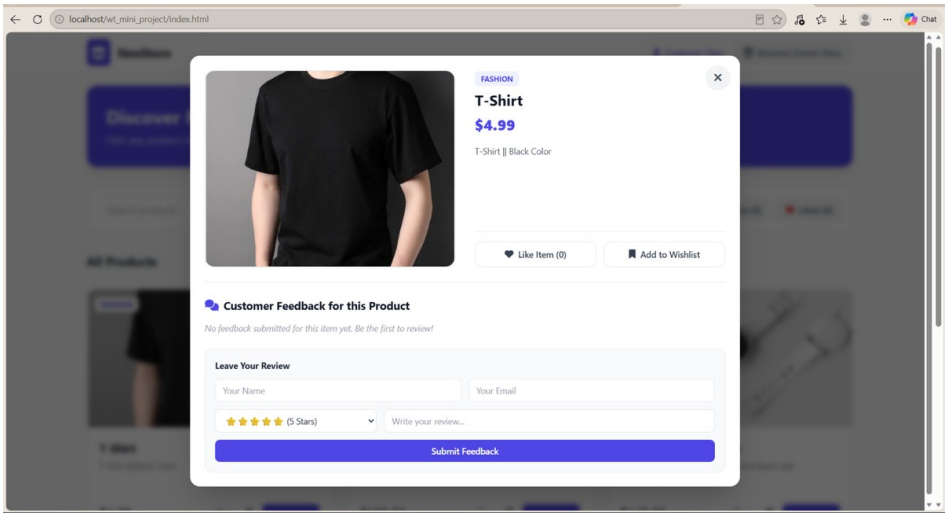


Figure 6.2: Product Detail Modal

6.3 BUSINESS OWNER DASHBOARD

The business-owner view contains the form used to add a new item to the catalog.

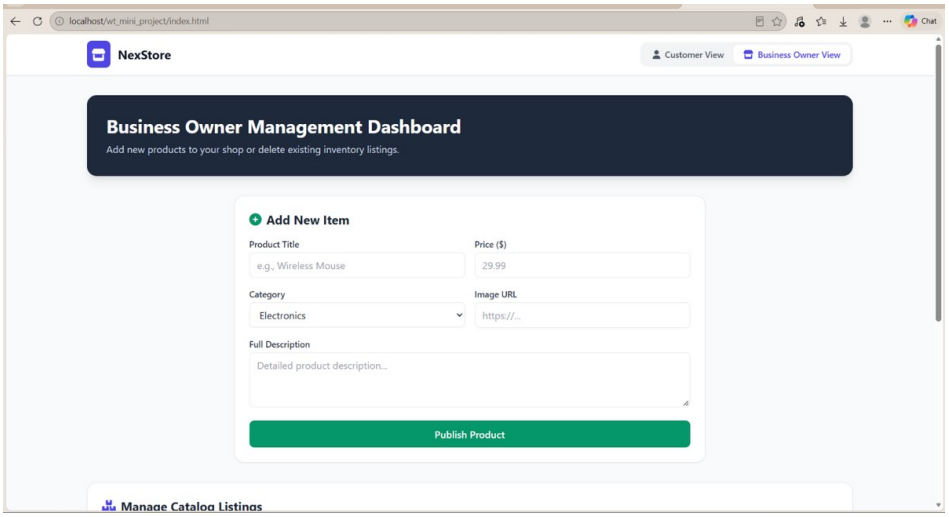


Figure 6.3: Business Owner Dashboard

6.4 SELLER INVENTORY AND DELETE ACTION

The Manage Catalog Listings area shows every product with a delete button.

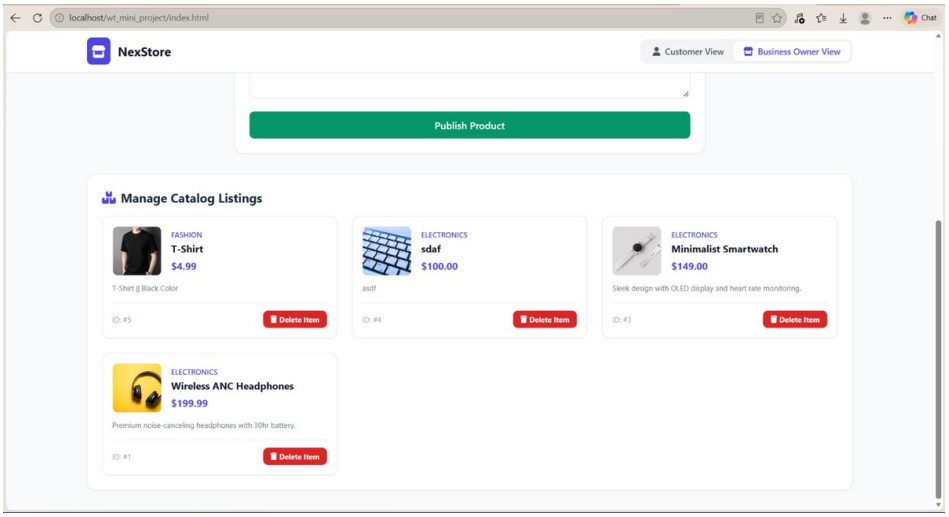


Figure 6.4: Seller Inventory and Delete Action

6.5 DATABASE / PHPMYADMIN – PRODUCTS TABLE

The shop_db database in phpMyAdmin shows the feedback, products, users and wishlist tables.

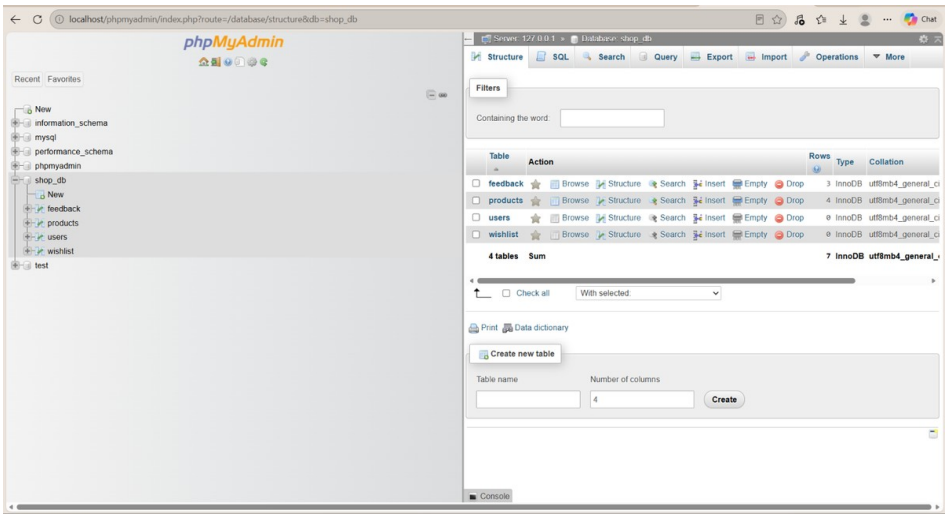


Figure 6.5: Database / phpMyAdmin – Products Table

CONCLUSION

NexStore demonstrates the practical integration of front-end and back-end Web Technology concepts in a compact e-commerce-style application. The project combines HTML5 for structure, Tailwind CSS and custom CSS for responsive presentation, JavaScript for dynamic behavior, PHP for server-side processing, PDO for database connectivity, and MySQL for data storage. The application provides a customer catalog, search and filters, product details, likes, wishlist interaction, and a business-owner dashboard for product administration.

The project also illustrates an important aspect of real-world development: interface functionality and database design must remain synchronized. In the supplied source, the feedback PHP code expects a product_id field while the supplied feedback table does not define it. Correcting this schema mismatch is necessary for reliable product-specific feedback persistence. Additional improvements such as authentication, secure authorization, persistent likes/wishlists, checkout, payment integration, image upload, inventory tracking, and stronger security controls can turn the academic prototype into a more complete e-commerce application.

Overall, the project meets the educational goal of demonstrating a database-driven Web Technology application with multiple interactive modules and clear separation of client-side and server-side responsibilities.

BIBLIOGRAPHY

[1] MDN Web Docs, “HTML, CSS, JavaScript and Fetch API documentation”. Available: https://developer.mozilla.org

[2] The PHP Group, “PHP Documentation: PHP language and PDO database access”. Available: https://www.php.net/manual

[3] Oracle, “MySQL Documentation: SQL, database and table design”. Available: https://dev.mysql.com/doc

[4] Tailwind Labs, “Tailwind CSS Documentation: utility-first CSS framework”. Available: https://tailwindcss.com/docs

[5] Fonticons, Inc., “Font Awesome Documentation: web icon library”. Available: https://fontawesome.com/docs